Reaching the memory roofline with Python: a fused-kernel lattice-Boltzmann solver on a laptop GPU

Walid Hemati
Independent researcher

Abstract

Lattice-Boltzmann (LB) solvers are bandwidth-bound: one D2Q9 update in single precision moves at least 72 B through DRAM, so the attainable speed is fixed by the memory system, not by the arithmetic. We ask whether a solver written and driven entirely from Python can sit on that roofline. Starting from an existing PyTorch cavity code, we build `pylbm`, a validated D2Q9/D3Q19 solver with BGK, TRT and MRT collisions, and climb an eight-rung optimization ladder from NumPy through eager and compiled PyTorch to fused stream–collide kernels in Triton, NVIDIA Warp, Numba-CUDA and hand-written CUDA C. On an RTX 3070 Laptop GPU the best fp32 D2Q9 kernel (Numba-CUDA fused) runs at 5,034 MLUPS, 101% of the measured device copy bandwidth of 361 GB/s; storing shifted populations $f_i - w_i$ in 16 bits halves the traffic and reaches 10,096 MLUPS. D3Q19 reaches 2,364 MLUPS (100% of copy bandwidth). On the same GPU and with the same metric, the best fp32 kernels run at 1.02 (D2Q9) and 1.00 (D3Q19) times the rate of the hand-optimised OpenCL code FluidX3D and 3.69 times that of the native kernel of the PyTorch framework Lettuce (D2Q9). The Triton, Warp and Numba kernels are generated from a single symbolic description, so all four toolchains execute identical arithmetic. The solver is checked against Ghia et al. (1982), shear-wave decay, Poiseuille flow, the Schäfer–Turek cylinder benchmark and 3D Taylor–Green decay; along the way we document three boundary-condition defects of the original code.

1 Introduction

The lattice-Boltzmann method (LBM) [1, 2] is a standard tool for weakly compressible flows. Its update is local and regular, which makes it a natural fit for GPUs [3–5], and its arithmetic intensity is low: a D2Q9 node reads nine populations and writes nine, about two floating-point operations per byte in fp32. On every current GPU this places LBM firmly under the memory roof of the roofline model [6]. The relevant figure of merit is therefore the fraction of the attainable memory bandwidth a code sustains, not its FLOP rate.

Production LB codes tend to be C++/CUDA [7] or use Python only to generate them [8]; Python LB frameworks built on tensor libraries [9, 10] trade some performance for flexibility. This paper starts from a small PyTorch lid-driven-cavity script (the earlier version of the present repository) that ran at a few hundred MLUPS and asks how far the same machine can be pushed without leaving Python as the driving language.

Contributions:

- A restructured, tested solver (`pylbm`) with a consistent axis convention, half-way bounce-back with Ladd's moving-wall correction [11], BGK, TRT and MRT collision, Guo forcing [12], a pure NumPy float64 reference implementation that acts as the correctness oracle, and a registry of interchangeable backends.

- An optimization ladder of eight rungs (NumPy, eager PyTorch with roll or gather streaming, `torch.compile`, and fused kernels in Triton, Warp, Numba-CUDA and CUDA C) plus memory-traffic variants: AA-pattern in-place streaming [13] and 16-bit storage of shifted populations [14]. Every rung reproduces the reference to round-off.
- A kernel generator that prints the same fused update for Triton, Warp and Numba, so that differences between those backends are purely due to the compilers.
- A roofline analysis against the measured copy bandwidth showing that Python-driven kernels reach the same hardware efficiency as hand-written CUDA C, and a same-GPU, same-metric head-to-head with FluidX3D [14] and Lettuce [9].
- Validation against five reference problems, and an account of the defects of the original code (mirrored axis, specular side walls, wrong lid density).

2 Background and related work

LBM on GPUs. Single-GPU LB implementations have tracked the memory bandwidth of each GPU generation. Tölke and Krafczyk [3] demonstrated 3D LBM on desktop GPUs with a D3Q13 model. Obrecht et al. [4] reported up to 512 MLUPS (LBGK) and 516 MLUPS (MRT) for D3Q19 in single precision on a GeForce GTX 295, reaching up to 86% of the maximum global-memory throughput. Mawson and Revell [5] reached 1036 MLUPS for D3Q19 BGK in single precision on a Tesla K20c. Lehmann et al. [14] report 5493 MLUPS for D3Q19 with FP32 storage on an RTX 3090 and 15 455 MLUPS on an A100 with FP32 arithmetic and FP16 storage of shifted populations.[1] Because these results come from different GPUs, we compare fractions of bandwidth, not MLUPS.

Memory layouts and streaming patterns. The two-buffer "pull" scheme reads $f_i(\mathbf{x}-\mathbf{c}_i)$ and writes $f_i(\mathbf{x})$, needing two copies of the populations. In-place schemes avoid the second copy: the AA pattern [13] alternates two kernels that read and write the same locations, and the esoteric twist [15] achieves the same with a single kernel. Wittmann et al. [16] compare propagation steps on CPUs and GPUs. Code-generation frameworks such as lbmpy [8] (used inside waLBerla [7]) derive optimized kernels symbolically; our generator follows the same idea at a much smaller scale.

Python toolchains. PyTorch [17] with `torch.compile` [18] lowers tensor programs to Triton [19] kernels. Triton, NVIDIA Warp [20], Numba [21] and CuPy [22] let one write GPU kernels from Python; CuPy compiles CUDA C at run time through NVRTC.

3 Method

3.1 Lattice Boltzmann equation

With populations $f_i(\mathbf{x},t)$ on a DdQq lattice (D2Q9 or D3Q19, weights w_i, lattice speed of sound $c_s^2 = 1/3$), one time step is

$$f_i(\mathbf{x}+\mathbf{c}_i, t+1) = f_i^\star(\mathbf{x},t) = f_i + \Omega_i(\mathbf{f}) + S_i, \tag{1}$$

with density $\rho = \sum_i f_i$ and velocity $\rho\mathbf{u} = \sum_i \mathbf{c}_i f_i + \mathbf{F}/2$. The equilibrium is the usual second-order polynomial $f_i^{\mathrm{eq}} = w_i\rho[1 + 3\,\mathbf{c}_i\cdot\mathbf{u} + \frac{9}{2}(\mathbf{c}_i\cdot\mathbf{u})^2 - \frac{3}{2}u^2]$. We implement three collision operators:

[1] Numbers from Table 5 and the conclusions of the arXiv version (2112.08926v2) of [14].

BGK [2, 23], $\Omega_i = -(f_i - f_i^{\mathrm{eq}})/\tau$, with kinematic viscosity $\nu = (\tau - \frac{1}{2})/3$; TRT [24], which relaxes the symmetric and antisymmetric parts of $f_i - f_i^{\mathrm{eq}}$ with rates $s_+ = 1/\tau$ and s_- chosen so that the "magic" parameter $\Lambda = (1/s_+ - \frac{1}{2})(1/s_- - \frac{1}{2}) = 3/16$; and MRT [25, 26] for D2Q9 in the orthogonal moment basis $(\rho, e, \varepsilon, j_x, q_x, j_y, q_y, p_{xx}, p_{xy})$ of Lallemand and Luo, with shear rates $1/\tau$, energy-flux rates from the same magic relation, and $s_e = 1.64$, $s_\varepsilon = 1.54$. Body forces enter through the Guo source term [12] $S_i = (1 - \frac{s}{2})w_i[3(\mathbf{c}_i - \mathbf{u})\cdot\mathbf{F} + 9(\mathbf{c}_i\cdot\mathbf{u})(\mathbf{c}_i\cdot\mathbf{F})]$, applied per relaxation rate for TRT and MRT.

3.2 Boundary conditions and conventions

Arrays are stored structure-of-arrays, $f[i, y, x]$ (or $f[i, z, y, x]$), with x the fastest index, y pointing up (row 0 is the bottom wall) and the lid on the top wall moving in $+x$. Walls are placed half-way between the last fluid node and the virtual solid node, so a cavity of n nodes has side length n and $\mathrm{Re} = Un/\nu$. A population leaving the fluid towards a wall returns in the opposite direction one step later,

$$f_{\bar{i}}(\mathbf{x}, t+1) = f_i^{\star}(\mathbf{x}, t) - 6w_i\rho_w\,\mathbf{c}_i\cdot\mathbf{u}_w, \tag{2}$$

where the moving-wall term is Ladd's momentum correction [11] with $\rho_w = 1$. At edges and corners the lid takes precedence. The cylinder benchmark additionally uses a velocity inlet (the same moving-wall rule with the parabolic profile as $\mathbf{u}_w$) and a pressure outlet that sets the incoming populations to $f_i^{\mathrm{eq}}(\rho = 1, \mathbf{u})$ with the velocity of the neighbouring column.

3.3 Defects of the original code

Running the original script through the same interface exposed three defects, all fixed in the new core. (i) Its streaming moves $c_y = +1$ populations towards decreasing row index while its plots use `origin="lower"`, so the lid that appears at the top is physically the $y = 0$ wall; the flow is the mirror image of the intended one. (ii) The side walls swap directions $5 \leftrightarrow 6$ and $8 \leftrightarrow 7$, which is a specular (free-slip) reflection, not bounce-back; no-slip was only enforced indirectly by zeroing $\mathbf{u}$ before the next equilibrium. (iii) The lid density was computed as $f_0 + f_1 + f_3 + f_2 + f_5 + f_6 + 2f_4$ instead of $f_0 + f_1 + f_3 + 2(f_4 + f_7 + f_8)$. With the correct mirror mapping, the original code deviates from Ghia's $\mathrm{Re} = 100$ profiles by $L_\infty = 0.0718$ (in units of the lid speed) on a 128^2 grid, against 0.0059 for the corrected solver on the same grid (section 5).

4 Implementation: the optimization ladder

4.1 One interface, one oracle

A `Problem` describes the box (per-axis periodic or walled), τ, collision operator, lid velocity and force; the general solvers additionally accept solid masks, an inlet profile and an outlet. Every backend implements `step(n)`, `populations()` (the post-collision state in canonical layout, as float64) and `set_populations()`. The NumPy reference evaluates the unfused algorithm in float64 with `np.roll` streaming and slice-assigned wall rules. The same array-level code runs on torch tensors, which gives the first PyTorch rungs for free.

4.2 The rungs

NumPy (float64). The reference: one array operation per population and term.

torch eager, roll. The reference code on CUDA tensors; about 150 kernel launches per step, so it is launch-bound at every size.

torch eager, gather. Streaming and all boundary rules collapse into a single `torch.take` with a precomputed index table. The table is produced by running the reference streaming routine on an array holding each entry's own index, so this rung is boundary-for-boundary identical to the reference by construction, and it supports arbitrary geometry (we use it for the cylinder benchmark).

torch.compile (original, roll, gather). The original script, and the roll and gather formulations, compiled by Inductor into Triton kernels.

Fused kernels. One kernel per time step (two for the AA pattern) that pulls the populations of a node, applies bounce-back, collides and stores, in Triton, Warp, Numba-CUDA and CUDA C (NVRTC via CuPy). Indices are 32-bit; the wall tests and periodic wraps reduce to comparisons of the node coordinates with the box edges, so no flag field is read and the traffic stays at the minimum.

4.3 A single kernel description for three Python DSLs

The Triton, Warp and Numba kernels are printed by a small generator (`pylbm/codegen.py`) from one description of the update. Each population becomes one select-then-load statement (the select chooses between the neighbour's slot and the own opposite slot for wall links, so exactly one load is issued), followed by straight-line moments, equilibria and collision with constants folded in, and one store per population. Triton operates on a block of nodes along x, Warp and Numba on one node per thread (Numba needs explicit `float32` literals to avoid silent promotion to fp64). The CUDA C kernel is written by hand with `#pragma unroll` loops over constant tables and specialised by preprocessor definitions; it does not share code with the generator, which makes it an independent check.

4.4 Memory-traffic variants

AA pattern. The two-buffer scheme holds $2q$ values per node. In the AA pattern [13] an "even" kernel reads $f_i(\mathbf{x})$ and writes the post-collision value into slot $\bar{\imath}$ of the same node; an "odd" kernel reads slot $\bar{\imath}$ of node $\mathbf{x} - \mathbf{c}_i$ and writes slot i of node $\mathbf{x} + \mathbf{c}_i$. Each node reads and writes exactly the locations it owns, so the update is race free in a single buffer. Wall links need care in the odd step on both sides: a link whose source is behind a wall reads the node's own slot i, and a link whose target is behind a wall writes back into the node's own slot $\bar{\imath}$ including the moving-wall term. Our AA kernels are bit-identical to the two-buffer kernels in fp32 (test `test_aa_pattern_bitwise_equal_to_two_buffer`). The traffic per update is unchanged, but the memory footprint halves.

16-bit storage. Lehmann et al. [14] show that FP32 arithmetic with 16-bit storage is accurate for many flows if the populations are stored shifted, as the deviation from the rest equilibrium, which recovers the bits otherwise spent on the constant w_i. We store $h_i = f_i - w_i$ in fp16 or bf16 and compute in fp32: the conversion costs a few instructions and the traffic drops to 36 B per D2Q9 node.

4.5 Traffic model and measurement

We use the minimal traffic model: every update reads and writes each population once, $2qs$ bytes for storage size s (72 B D2Q9/fp32, 36 B D2Q9/16-bit, 152 B D3Q19/fp32, 76 B D3Q19/16-bit). Boundary handling adds no traffic in the fused kernels. Effective bandwidth is MLUPS times this byte count, so it is never inflated. The ceiling is the bandwidth of a device-to-device copy measured at the start of each benchmark run with a 1 GiB `torch copy_` (read plus write counted): 361 GB/s on the NVIDIA GeForce RTX 3070 Laptop GPU, against a specification value of 448 GB/s. The resulting fp32 D2Q9 roofline is 5,008 MLUPS, the 16-bit roofline 10,016 MLUPS and the D3Q19 fp32 roofline 2,372 MLUPS. The benchmark problem is the lid-driven cavity (all walls active). Timings follow a fixed protocol: warm-up including JIT compilation, device synchronisation around every timed region, five repeats of at least 0.5 s each, and the median reported with min/max.

5 Validation

Backend equivalence. After 100–200 steps on small cavities, channels and periodic boxes (BGK, TRT, MRT, with and without force, 2D and 3D), every fp32 backend agrees with the float64 reference to 1.7e-06 in the populations, the fp16 variants to 3.6e-05 and the bf16 variant to 2.1e-04 (15 backend variants; `tests/test_equivalence.py`).

Lid-driven cavity. Figure 1 compares the centreline velocities with the tabulated values of Ghia et al. [27] at Re = 100, 400 and 1000 (lid speed $U = 0.1$). Errors are computed at the 15 interior table points per profile (the Re = 400 v value at $x = 0.9063$, a widely reported misprint, is excluded) and given in units of U in table 1. They are at the level of the differences between Ghia's 129^2 multigrid solution and modern high-resolution solutions, and do not decrease further with grid refinement at Re = 100 (0.0060, 0.0059, 0.0072 on 64^2, 128^2, 256^2). TRT and MRT agree with BGK at Re = 1000, and the AA-pattern kernel reproduces the two-buffer result exactly. With shifted fp16 storage the Re = 100 error is unchanged ($L_\infty = 0.0050$) and the Re = 1000 error grows from 0.0080 to 0.0141, still about 1% of the lid speed; the velocity field then stagnates at a round-off noise floor instead of converging to the fp32 tolerance. bf16, with an 8-bit significand, is clearly not accurate enough ($L_\infty = 0.0653$ at Re = 1000).

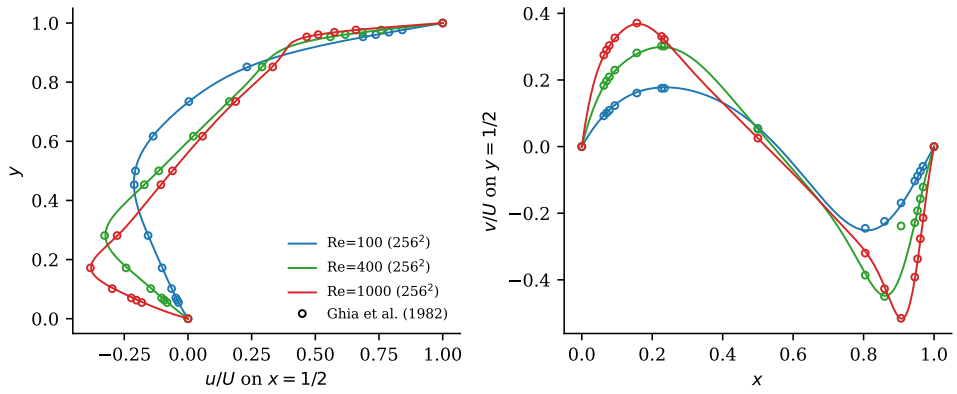


Figure 1: Lid-driven cavity centreline profiles (lines, finest grid per Re) against Ghia et al. (1982) (circles). The outlying Re = 400 point at $x = 0.9063$ is the misprint excluded from the error norms.

Table 1: Cavity centreline errors against Ghia et al. (1982) in units of the lid speed (L_2: RMS over the interior table points of both profiles; L_∞: maximum).

case	solver	L_2	L_∞
Re 100, 128^2	Triton fused, BGK	0.0029	0.0059
Re 400, 256^2	Triton fused, BGK	0.0024	0.0047
Re 1000, 256^2	Triton fused, BGK	0.0040	0.0080
Re 1000, 256^2	Triton fused, TRT	0.0040	0.0074
Re 1000, 256^2	Triton fused, MRT	0.0040	0.0082
Re 1000, 256^2	Triton fused AA, BGK	0.0040	0.0080
Re 100, 128^2	Triton, fp16 storage	0.0025	0.0050
Re 100, 128^2	Triton, bf16 storage	0.0161	0.0285
Re 1000, 256^2	Triton, fp16 storage	0.0079	0.0141
Re 1000, 256^2	CUDA C, AA + fp16	0.0072	0.0129
Re 1000, 256^2	Triton, bf16 storage	0.0413	0.0653
Re 100, 128^2	original code	0.0250	0.0718

Shear-wave decay. A sinusoidal shear wave $u_x = A\sin(2\pi y/n)$ in a periodic 128^2 box decays as $\exp(-\nu k^2 t)$. Fitting the decay gives the measured viscosity (fig. 2). For $\tau \in [0.52, 1.95]$ the largest relative deviation from $(\tau - \frac{1}{2})/3$ is 0.17%, and 0.09% for $0.6 \le \tau \le 1.6$; the remaining deviation is the known higher-order truncation error that grows towards both ends of the τ range.

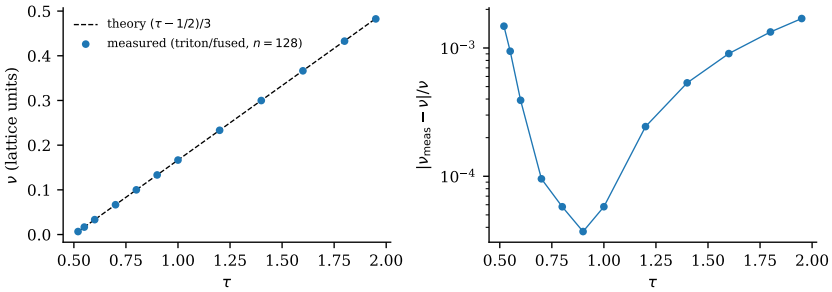


Figure 2: Shear-wave decay: measured against theoretical viscosity (left) and relative error (right).

Poiseuille flow. A body-force driven channel of width H at fixed $\tau = 0.8$ and fixed peak velocity converges to the analytic parabola with order 2.000 for BGK (relative L_2 error 1.7e-04 at $H = 64$), and TRT with $\Lambda = 3/16$ reproduces the parabola to round-off (largest error 4e-12), as expected for half-way bounce-back with the magic parameter (fig. 3). The convergence study uses the float64 reference: in fp32 the tiny force increments ($\sim 10^{-6}$ per step) are partly lost to round-off when added to populations of order 0.1, which limits the $H = 32$ error to 1.1e-03 against 7.0e-04 in float64.

Cylinder in a channel. For the Schäfer–Turek benchmark 2D-2 [28] (channel 2.2×0.41, cylinder diameter 0.1 at $(0.2, 0.2)$, parabolic inflow with mean velocity $\bar{U}$, $\mathrm{Re} = \bar{U}D/\nu = 100$) we use $D = 40$ nodes (880×164 lattice, $\bar{U} = 0.05$, $\tau = 0.560$, TRT), a staircase cylinder and momentum-exchange

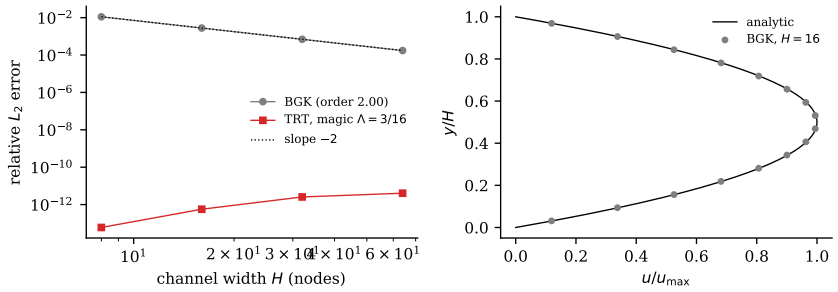


Figure 3: Poiseuille channel: grid convergence (left) and profile (right).

forces. The flow settles into periodic vortex shedding (fig. 4) with Strouhal number $\mathrm{St} = fD/\bar{U} = 0.2996$, maximum drag coefficient $c_{D,\max} = 3.312$ and maximum lift coefficient $c_{L,\max} = 1.005$. The benchmark's reference intervals are $[0.2950, 0.3050]$, $[3.22, 3.24]$ and $[0.99, 1.01]$; the drag is over-predicted, as expected for a staircase cylinder with simple bounce-back at this resolution.

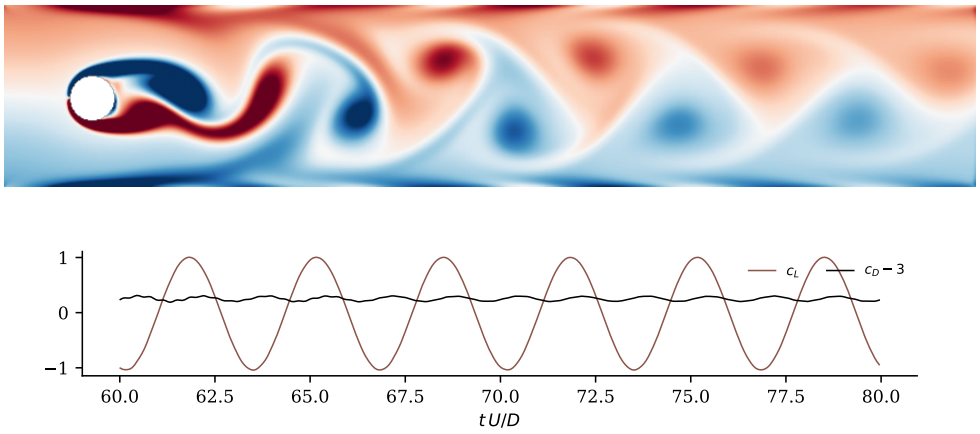


Figure 4: Schäfer–Turek 2D-2 at $\mathrm{Re} = 100$: vorticity (units of $\bar{U}/D$) and the lift and drag coefficients over the measurement window.

3D. A 2D Taylor–Green vortex embedded in each coordinate plane of a periodic 64^3 D3Q19 box decays with the expected viscosity to within 0.11% (Triton and CUDA C), and the 3D lid-driven cavity of every 3D backend matches the reference.

6 Performance

Figure 5 and table 2 show the ladder for the lid-driven cavity. The fused kernels are evaluated at the largest benchmark size; the slow rungs at the sizes where they finish in reasonable time.

From the original code to the roofline. The original script runs at 458 MLUPS (9.1% of copy bandwidth): its `torch.compile`d functions are separated by Python-level boundary updates on an array-of-structures layout, so populations make several round trips through DRAM per step. Compiling the clean roll formulation lets Inductor fuse the step and gives 4,983 MLUPS (100%):

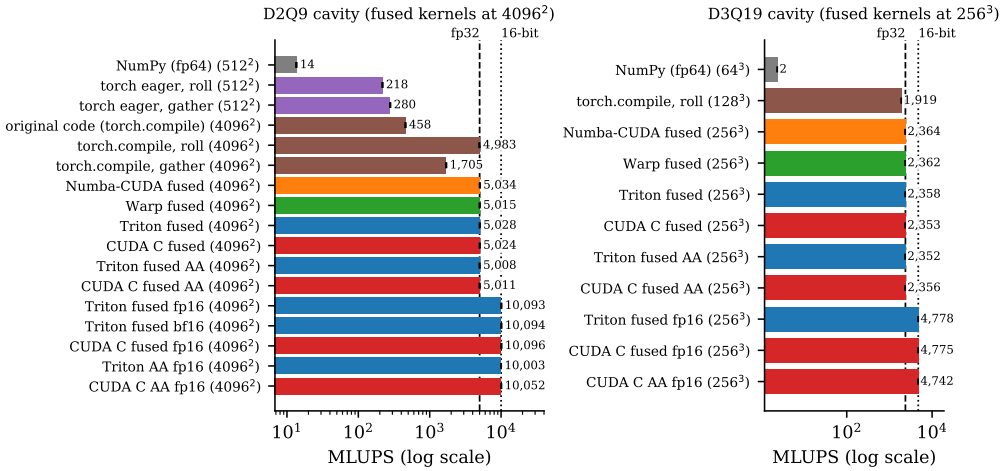


Figure 5: The optimization ladder (median, whiskers min/max). Dashed: fp32 roofline at the measured copy bandwidth (361 GB/s); dotted: roofline for 16-bit storage.

Table 2: Lid-driven cavity throughput on the NVIDIA GeForce RTX 3070 Laptop GPU (median of 5). B/LU: bytes per lattice update in the minimal traffic model; % copy and % spec: effective bandwidth as a fraction of the measured copy bandwidth and of the 448 GB/s specification.

	rung	size	MLUPS	B/LU	GB/s	% copy	% spec
D2Q9	NumPy (fp64)	512 × 512	14	144	2	1	0
D2Q9	torch eager, roll	512 × 512	218	72	16	4	4
D2Q9	torch eager, gather	512 × 512	280	72	20	6	5
D2Q9	original code (torch.compile)	4096 × 4096	458	72	33	9	7
D2Q9	torch.compile, roll	4096 × 4096	4,983	72	359	100	80
D2Q9	torch.compile, gather	4096 × 4096	1,705	72	123	34	27
D2Q9	Numba-CUDA fused	4096 × 4096	5,034	72	362	101	81
D2Q9	Warp fused	4096 × 4096	5,015	72	361	100	81
D2Q9	Triton fused	4096 × 4096	5,028	72	362	100	81
D2Q9	CUDA C fused	4096 × 4096	5,024	72	362	100	81
D2Q9	Triton fused AA	4096 × 4096	5,008	72	361	100	80
D2Q9	CUDA C fused AA	4096 × 4096	5,011	72	361	100	81
D2Q9	Triton fused fp16	4096 × 4096	10,093	36	363	101	81
D2Q9	Triton fused bf16	4096 × 4096	10,094	36	363	101	81
D2Q9	CUDA C fused fp16	4096 × 4096	10,096	36	363	101	81
D2Q9	Triton AA fp16	4096 × 4096	10,003	36	360	100	80
D2Q9	CUDA C AA fp16	4096 × 4096	10,052	36	362	100	81
D3Q19	NumPy (fp64)	64 × 64 × 64	2	304	1	0	0
D3Q19	torch.compile, roll	128 × 128 × 128	1,919	152	292	81	65
D3Q19	Numba-CUDA fused	256 × 256 × 256	2,364	152	359	100	80
D3Q19	Warp fused	256 × 256 × 256	2,362	152	359	100	80
D3Q19	Triton fused	256 × 256 × 256	2,358	152	358	99	80
D3Q19	CUDA C fused	256 × 256 × 256	2,353	152	358	99	80
D3Q19	Triton fused AA	256 × 256 × 256	2,352	152	357	99	80
D3Q19	CUDA C fused AA	256 × 256 × 256	2,356	152	358	99	80
D3Q19	Triton fused fp16	256 × 256 × 256	4,778	76	363	101	81
D3Q19	CUDA C fused fp16	256 × 256 × 256	4,775	76	363	101	81
D3Q19	CUDA C AA fp16	256 × 256 × 256	4,742	76	360	100	80

for fp32 D2Q9, `torch.compile` alone already reaches the roofline. The hand-written fused kernels match it rather than beat it, 5,028 MLUPS for Triton (100%), 5,015 for Warp (100%), 5,034 for Numba (101%) and 5,024 for CUDA C (100%). They are needed where Inductor stops: in D3Q19 the compiled roll formulation reaches 81% of copy bandwidth (table 2), and 16-bit storage and the AA pattern change the data layout, which only the fused kernels implement. The best fp32 variant is 11.0× faster than the original code, and the best 16-bit variant 22.1×. Values slightly above 100% of the copy bandwidth are possible because a plain copy does not saturate DRAM perfectly either; the specification bandwidth is a hard upper bound that no variant exceeds.

Collision operators are free. At these intensities the arithmetic is hidden behind memory traffic: the Triton kernel reaches 100% (TRT) and 100% (MRT) of copy bandwidth, against 100% for BGK.

Problem size. Figure 6 shows throughput against lattice size. The fused kernels reach the plateau from about 10^6 nodes; below that the per-launch overhead of the Python runtime (tens of microseconds per kernel launch through Triton's and Warp's launchers) dominates. D3Q19 reaches 2,364 MLUPS (100% of copy bandwidth) in fp32 and 4,778 MLUPS (101%) with 16-bit storage.

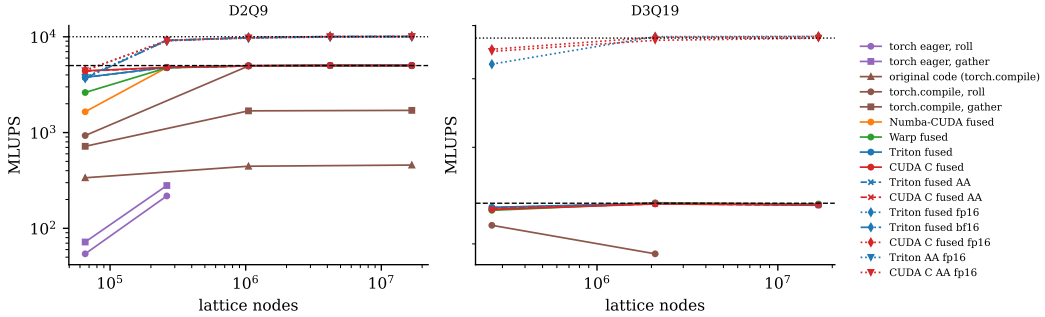


Figure 6: Throughput against lattice size for D2Q9 (left) and D3Q19 (right). Solid: two-buffer fp32; dashed: AA pattern; dotted: 16-bit storage. Horizontal lines: rooflines.

6.1 Head-to-head with FluidX3D and Lettuce on the same GPU

Published efficiencies come from other GPUs (Obrecht et al. [4]: up to 86% of a GTX 295's global-memory throughput; Lehmann et al. [14]: 5493 MLUPS × 152 B, about 89% of an RTX 3090's 936 GB/s). We therefore ran two established codes on our GPU and scored them with our metric and timing protocol (section 4.5). FluidX3D (OpenCL, commit f0cbdeb; in-place Esoteric-Pull streaming [29]) runs its own benchmark case, a periodic box, with SRT (= BGK), FP32 arithmetic and FP32, FP16S or FP16C storage; we replaced only its benchmark `main_setup` by a timing harness. Lettuce [9] (0.3.1.dev1, commit 88a6559, PyTorch 2.13.0+cu130) runs the case of its `lettuce benchmark` command (periodic Taylor–Green vortex, BGK, fp32) on its PyTorch path and with its generated native CUDA kernel. pylbm keeps the cavity, whose walls only add work; FluidX3D also reads a flag byte per cell that the model does not credit, and both counterparts synchronise host and device after every step, which matters only on small lattices.

In fp32 (fig. 7), FluidX3D reaches 4,943 MLUPS (99% of copy bandwidth) for D2Q9 at 4096^2 and 2,350 MLUPS (99%) for D3Q19 at 256^3; pylbm's best fp32 kernel runs at 1.02 and 1.00 times these rates. FP16S reaches 97% and 98% (pylbm's best 16-bit kernel: 1.04 and 1.02 times that),

FP16C, whose format is converted in software, 74% and 76%. Lettuce's native kernel reaches 27% (D2Q9) and 14% (D3Q19, 192^3), its PyTorch path, which streams with one `torch.roll` per population like our eager rung, 3% and 3%; in D2Q9 pylbm is 3.69 and 36 times faster. FluidX3D's license permits use for public research and does not restrict publishing benchmark results; it requires that results of altered versions be published with the altered source (our harness is generated by `bench/counterparts.py`) and asks publications to cite [14, 29–34].

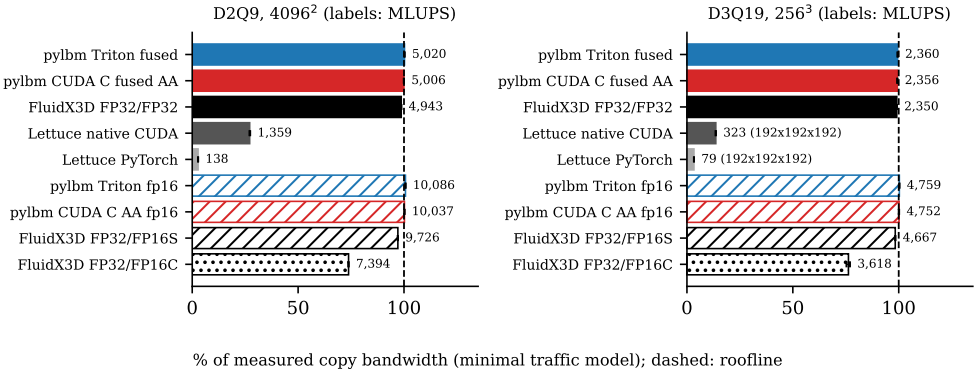


Figure 7: Same-GPU head-to-head (median, whiskers min/max) at the largest lattice of the run. Colours: pylbm (Triton blue, CUDA C red); FluidX3D black; Lettuce grey. Hatched: 16-bit storage. Bars give the fraction of the measured copy bandwidth in the minimal traffic model, labels the MLUPS.

7 Discussion

What Python costs. At the kernel level nothing: once the update is a single fused kernel, Triton, Warp, Numba and NVRTC-compiled CUDA C all land on the roofline, at 1.02 (D2Q9) and 1.00 (D3Q19) times FluidX3D's fp32 rate on the same GPU (section 6.1). Python's cost is launch overhead (visible at small sizes) and the temptation to write the algorithm as many small array operations, which limits the eager rungs and Lettuce's PyTorch path. `torch.compile` removes that cost automatically for the 2D roll formulation, but not for the gather formulation, where the index table doubles the traffic, and only partly in 3D (81%).

Threats to validity. All measurements come from a single RTX 3070 Laptop GPU. Laptop GPUs adjust clocks with temperature and power; we record the SM clock and temperature with every run, but sustained runs may throttle. During development the GPU was shared with other jobs; publishable numbers come from a serial run on an otherwise idle GPU. The copy-bandwidth ceiling is itself a measurement (361 GB/s, 1 GiB buffers) and a kernel with a better access pattern can exceed it slightly; we therefore also report the fraction of the specification bandwidth. CUDA C is compiled with NVRTC through CuPy, not `nvcc`; both use the same NVVM back end, but default flags differ. The minimal traffic model ignores the (cached) re-reads of neighbouring rows and the boundary branches, so it is a lower bound on actual DRAM traffic. The benchmark is a lid-driven cavity: problems with complex geometry need a flag field or index lists and move more data. The counterparts run their own cases with their default settings; we did not tune them.

Limitations and future work. The fused kernels handle box geometries (walls, periodicity, moving lid, body force); the cylinder benchmark uses the gather backend. MRT is implemented for D2Q9 only. Only one GPU was available, so multi-GPU domain decomposition with halo exchange is left for future work; we make no scaling claims.

8 Conclusion

A lattice-Boltzmann solver written and driven from Python reaches the memory roofline of a laptop GPU: fused kernels written in Triton, Warp, Numba or CUDA C all sustain close to the measured copy bandwidth, and 16-bit storage of shifted populations nearly doubles the update rate at a moderate accuracy cost (fp16) on the cavity benchmark. The path from a few hundred to several thousand MLUPS was mostly about data movement, not about the language. On the same GPU the fused kernels run at 1.02 (D2Q9) and 1.00 (D3Q19) times the fp32 rate of the hand-optimised FluidX3D (section 6.1).

Code availability

Code, benchmark data and the scripts that generate every number and figure in this paper are available at `https://github.com/walidhemati/pylbm` under the MIT license. `bench/run.py --full` reproduces the performance data, `bench/counterparts.py --full` the head-to-head, `pylbm validate` the validation data, and `figures/make_figures.py` the figures and `paper/results.tex`.

References

[1] Timm Krüger, Halim Kusumaatmaja, Alexandr Kuzmin, Orest Shardt, Goncalo Silva, and Erlend Magnus Viggen. *The Lattice Boltzmann Method: Principles and Practice*. Graduate Texts in Physics. Springer International Publishing, Cham, 2017. ISBN 978-3-319-44647-9. doi: 10.1007/978-3-319-44649-3.

[2] Y. H. Qian, D. d'Humières, and P. Lallemand. Lattice BGK models for Navier-Stokes equation. *Europhysics Letters*, 17(6):479–484, 1992. doi: 10.1209/0295-5075/17/6/001.

[3] J. Tölke and M. Krafczyk. TeraFLOP computing on a desktop PC with GPUs for 3D CFD. *International Journal of Computational Fluid Dynamics*, 22 (7):443–456, 2008. doi: 10.1080/10618560802238275.

[4] Christian Obrecht, Frédéric Kuznik, Bernard Tourancheau, and Jean-Jacques Roux. A new approach to the lattice Boltzmann method for graphics processing units. *Computers & Mathematics with Applications*, 61(12):3628–3638, 2011. doi: 10.1016/j.camwa.2010.01.054.

[5] Mark J. Mawson and Alistair J. Revell. Memory transfer optimization for a lattice Boltzmann solver on Kepler architecture nVidia GPUs. *Computer Physics Communications*, 185(10):2566–2574, 2014. doi: 10.1016/j.cpc.2014.06.003.

[6] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[7] Martin Bauer, Sebastian Eibl, Christian Godenschwager, Nils Kohl, Michael Kuron, Christoph Rettinger, Florian Schornbaum, Christoph Schwarzmeier, Dominik Thönnes, Harald Köstler, and Ulrich Rüde. waLBerla: A block-structured high-performance framework for multiphysics simulations. *Computers & Mathematics with Applications*, 81:478–501, 2021. doi: 10.1016/j.camwa.2020.01.007.

[8] Martin Bauer, Harald Köstler, and Ulrich Rüde. lbmpy: Automatic code generation for efficient parallel lattice Boltzmann methods. *Journal of Computational Science*, 49:101269, 2021. doi: 10.1016/j.jocs.2020.101269.

[9] Mario Christopher Bedrunka, Dominik Wilde, Martin Kliemank, Dirk Reith, Holger Foysi, and Andreas Krämer. Lettuce: PyTorch-based lattice Boltzmann framework. In Heike Jagode, Hartwig Anzt, Hatem Ltaief, and Piotr Luszczek, editors, *High Performance Computing: ISC High Performance Digital 2021 International Workshops, Frankfurt am Main, Germany, June 24 – July 2, 2021, Revised Selected Papers*, volume 12761 of *Lecture Notes in Computer Science*, pages 40–55, Cham, 2021. Springer. doi: 10.1007/978-3-030-90539-2_3.

[10] Mohammadmehdi Ataei and Hesam Salehipour. XLB: A differentiable massively parallel lattice Boltzmann library in Python. *Computer Physics Communications*, 300:109187, 2024. doi: 10.1016/

j.cpc.2024.109187.

[11] Anthony J. C. Ladd. Numerical simulations of particulate suspensions via a discretized Boltzmann equation. Part 1. Theoretical foundation. *Journal of Fluid Mechanics*, 271:285–309, 1994. doi: 10.1017/S0022112094001771.

[12] Zhaoli Guo, Chuguang Zheng, and Baochang Shi. Discrete lattice effects on the forcing term in the lattice Boltzmann method. *Physical Review E*, 65(4): 046308, 2002. doi: 10.1103/PhysRevE.65.046308.

[13] P. Bailey, J. Myre, S. D. C. Walsh, D. J. Lilja, and M. O. Saar. Accelerating lattice Boltzmann fluid flow simulations using graphics processors. In *2009 International Conference on Parallel Processing (ICPP)*, pages 550–557. IEEE, 2009. doi: 10.1109/ICPP.2009.38.

[14] Moritz Lehmann, Mathias J. Krause, Giorgio Amati, Marcello Sega, Jens Harting, and Stephan Gekle. Accuracy and performance of the lattice Boltzmann method with 64-bit, 32-bit, and customized 16-bit number formats. *Physical Review E*, 106(1):015308, 2022. doi: 10.1103/PhysRevE.106.015308.

[15] Martin Geier and Martin Schönherr. Esoteric twist: An efficient in-place streaming algorithmus for the lattice Boltzmann method on massively parallel hardware. *Computation*, 5(2):19, 2017. doi: 10.3390/computation5020019.

[16] Markus Wittmann, Thomas Zeiser, Georg Hager, and Gerhard Wellein. Comparison of different propagation steps for lattice Boltzmann methods. *Computers & Mathematics with Applications*, 65(6):924–935, 2013. doi: 10.1016/j.camwa.2012.05.002.

[17] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

[18] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pages 929–947. ACM, 2024. doi: 10.1145/3620665.3640366.

[19] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.

[20] Miles Macklin. Warp: A high-performance Python framework for GPU simulation and graphics. NVIDIA GPU Technology Conference (GTC), March 2022. URL https://github.com/NVIDIA/warp.

[21] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC '15)*, pages 1–6. ACM, 2015. doi: 10.1145/2833157.2833162.

[22] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.

[23] P. L. Bhatnagar, E. P. Gross, and M. Krook. A model for collision processes in gases. I. small amplitude processes in charged and neutral one-component systems. *Physical Review*, 94(3):511–525, 1954. doi: 10.1103/PhysRev.94.511.

[24] Irina Ginzburg, Frederik Verhaeghe, and Dominique d'Humières. Two-relaxation-time lattice Boltzmann scheme: About parametrization, velocity, pressure and mixed boundary conditions. *Communications in Computational Physics*, 3(2):427–478, 2008. doi: 10.4208/cicp.2008.v3.p427.

[25] Pierre Lallemand and Li-Shi Luo. Theory of the lattice Boltzmann method: Dispersion, dissipation, isotropy, Galilean invariance, and stability. *Physical Review E*, 61(6):6546–6562, 2000. doi: 10.1103/PhysRevE.61.6546.

[26] Dominique d'Humières, Irina Ginzburg, Manfred Krafczyk, Pierre Lallemand, and Li-Shi Luo. Multiple-relaxation-time lattice Boltzmann models in three dimensions. *Philosophical Transactions of*

the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences, 360(1792): 437–451, 2002. doi: 10.1098/rsta.2001.0955.

[27] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982. doi: 10.1016/0021-9991(82)90058-4.

[28] M. Schäfer and S. Turek. Benchmark computations of laminar flow around a cylinder. In Ernst Heinrich Hirschel, editor, *Flow Simulation with High-Performance Computers II*, volume 52 of *Notes on Numerical Fluid Mechanics*, pages 547–566. Vieweg+Teubner Verlag, Wiesbaden, 1996. doi: 10.1007/978-3-322-89849-4_39. With support by F. Durst, E. Krause and R. Rannacher.

[29] Moritz Lehmann. Esoteric pull and esoteric push: Two simple in-place streaming schemes for the lattice Boltzmann method on GPUs. *Computation*, 10 (6):92, 2022. doi: 10.3390/computation10060092.

[30] Moritz Lehmann. *Computational study of microplastic transport at the water-air interface with a memory-optimized lattice Boltzmann method*. PhD thesis, University of Bayreuth, 2023.

[31] Moritz Lehmann. Combined scientific CFD simulation and interactive raytracing with OpenCL. In *International Workshop on OpenCL (IWOCL'22)*, pages 1–2. ACM, 2022. doi: 10.1145/3529538.3529542.

[32] Moritz Lehmann, Lisa Marie Oehlschlägel, Fabian P. Häusl, Andreas Held, and Stephan Gekle. Ejection of marine microplastics by raindrops: a computational and experimental study. *Microplastics and Nanoplastics*, 1(1):18, 2021. doi: 10.1186/s43591-021-00018-8.

[33] Moritz Lehmann. High performance free surface LBM on GPUs. Master's thesis, University of Bayreuth, 2019.

[34] Moritz Lehmann and Stephan Gekle. Analytic solution to the piecewise linear interface construction problem and its application in curvature calculation for volume-of-fluid simulation codes. *Computation*, 10(2):21, 2022. doi: 10.3390/computation10020021.